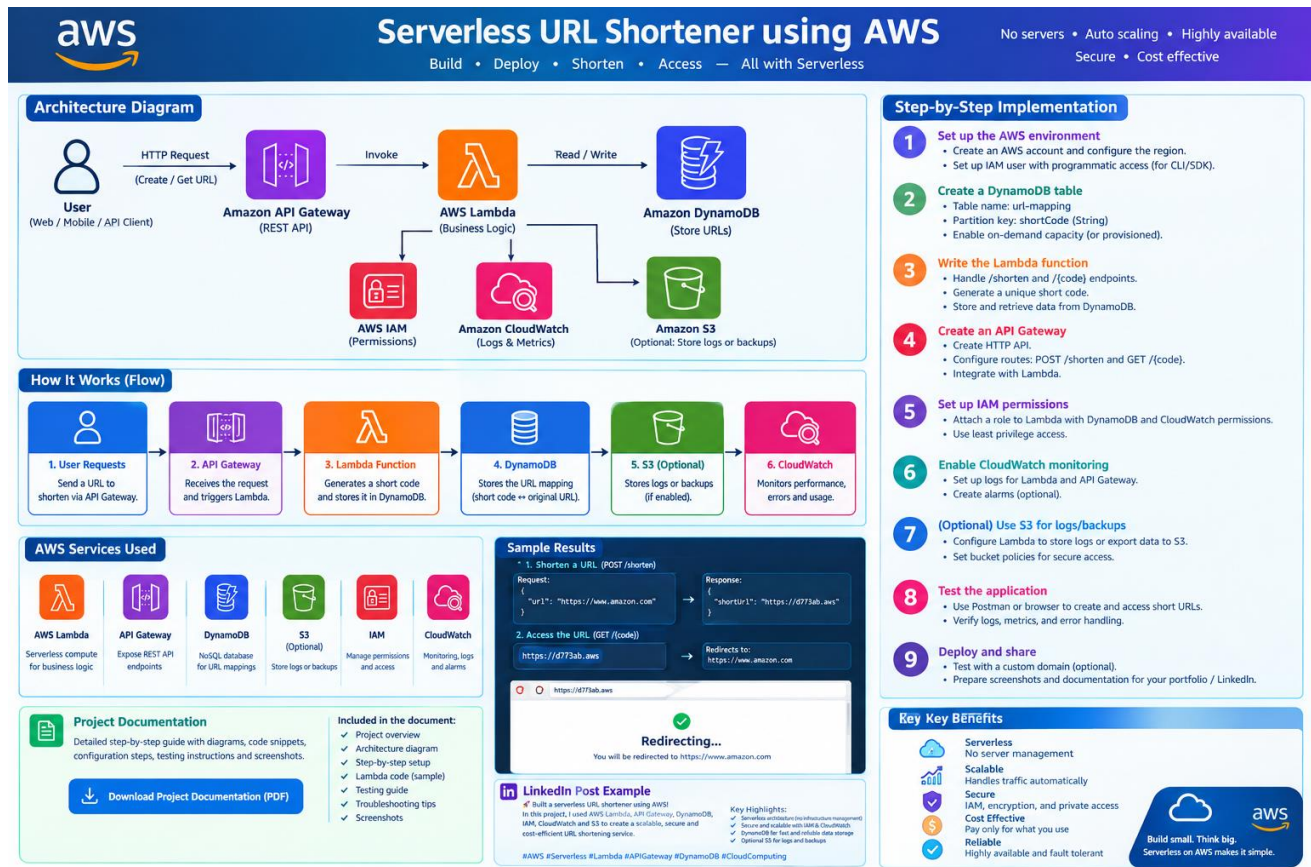


SERVERLESS URL SHORTENER USING AWS

Project documentation • Architecture • Step-by-step build • Code • Testing • LinkedIn



1. Project overview

This project creates a URL-shortening API without managing servers. A client sends a long URL to API Gateway. API Gateway invokes Lambda, which validates the request, creates a short code, and stores the mapping in DynamoDB. Visiting the short URL looks up the code and returns an HTTP redirect.

Core services: API Gateway, Lambda, DynamoDB, IAM, and CloudWatch. S3 is optional for static assets, exports, or archived files; it is not required for the core shortening workflow.

2. Architecture and request flow

Create: Client → API Gateway POST /shorten → Lambda → DynamoDB PutItem → JSON response containing a short URL.

Redirect: Browser → API Gateway GET /{code} → Lambda → DynamoDB GetItem → HTTP 302 response with a Location header.

IAM grants Lambda narrowly scoped permissions. CloudWatch collects Lambda logs and metrics. Configure API Gateway access logging separately if request-level logs are needed.

3. Step-by-step implementation

Step 1 — Choose a Region. Create a DynamoDB table named url-mappings with a string partition key named code. On-demand capacity is convenient for a learning project.

Step 2 — Create a Lambda execution role. Attach `AWSLambdaBasicExecutionRole`. Add a custom policy allowing `dynamodb:PutItem` and `dynamodb:GetItem` only on the `url-mappings` table ARN.

Step 3 — Create a Lambda function using a supported Python runtime. Set `TABLE_NAME=url-mappings` and `BASE_URL` to the deployed API base URL.

Step 4 — Create an API Gateway HTTP API. Add `POST /shorten` and `GET /{code}`; integrate both routes with the Lambda function. Ensure API Gateway has permission to invoke Lambda.

Step 5 — Deploy the API. Copy its invoke URL and set `BASE_URL` to that base URL, without a trailing slash. Update the Lambda environment variable and redeploy if needed.

Step 6 — Test `POST /shorten` with JSON such as `{"url":"https://example.com/article"}`. Expect HTTP 201 with a code and `shortUrl`. Open the short URL; expect HTTP 302 and a `Location` header.

Step 7 — Configure CloudWatch. Review the Lambda log group; create alarms for errors, throttles, or duration. Configure API Gateway access logs if needed. Avoid logging secrets.

Step 8 — Optional S3. Create a private bucket with Block Public Access and encryption enabled. Use it for static assets or exports, and grant only the required bucket permissions.

Step 9 — Harden the service. Validate URLs, allow only `http/https`, handle unknown codes, limit request sizes, and consider abuse prevention and throttling before making it public.

4. Sample Lambda code (Python)

```
import json, os, secrets, string
from urllib.parse import urlparse
import boto3
from botocore.exceptions import ClientError

table = boto3.resource("dynamodb").Table(os.environ["TABLE_NAME"])
BASE_URL = os.environ.get("BASE_URL", "").rstrip("/")
ALPHABET = string.ascii_letters + string.digits

def make_code(length=7):
    return "".join(secrets.choice(ALPHABET) for _ in range(length))

def reply(status, body=None, headers=None):
    return {
        "statusCode": status,
        "headers": {"content-type": "application/json", **(headers or {})},
        "body": json.dumps(body) if body is not None else ""
    }

def lambda_handler(event, context):
    method = event.get("requestContext", {}).get("http", {}).get("method", "")
    path = event.get("rawPath", "/")

    if method == "POST" and path.endswith("/shorten"):
        try:
            body = json.loads(event.get("body") or "{}")
            original = body.get("url", "").strip()
            parsed = urlparse(original)
            if parsed.scheme not in ("http", "https") or not parsed.netloc:
                return reply(400, {"error": "Provide a valid http or https URL."})
        except (json.JSONDecodeError, AttributeError):
            return reply(400, {"error": "Request body must be valid JSON."})

    for _ in range(5):
        code = make_code()
        try:
            table.put_item(
                Item={"code": code, "originalUrl": original},
                ConditionExpression="attribute_not_exists(code)"
            )
            short_url = f"{BASE_URL}/{code}" if BASE_URL else f"/{code}"
            return reply(201, {"code": code, "shortUrl": short_url})
        except ClientError as exc:
```

```

        if exc.response["Error"]["Code"] != "ConditionalCheckFailedException":
            raise
        return reply(500, {"error": "Could not allocate a short code."})

if method == "GET":
    code = path.rstrip("/").split("/")[-1]
    if not code or code == "shorten":
        return reply(404, {"error": "Not found"})
    item = table.get_item(Key={"code": code}).get("Item")
    if not item:
        return reply(404, {"error": "Short URL not found"})
    return {"statusCode": 302,
            "headers": {"Location": item["originalUrl"], "Cache-Control": "no-store"},
            "body": ""}

return reply(404, {"error": "Route not found"})

```

Assumptions: API Gateway HTTP API with payload format 2.0 and the routes above. For production, add automated tests, stronger validation, abuse controls, and error handling.

5. IAM and security

Use the Lambda execution role; do not put AWS access keys in code. Scope DynamoDB permissions to the exact table ARN. Keep the DynamoDB table and S3 bucket private. Do not store secrets in shortened URLs.

Short codes are identifiers, not authentication. Consider expiry (DynamoDB TTL), rate limits, abuse reporting, and domain reputation controls for a public service.

6. Test checklist

- ☐ POST /shorten with a valid HTTPS URL returns 201.
- ☐ Invalid URL or malformed JSON returns 400.
- ☐ GET /{code} returns 302 and the correct Location header.
- ☐ Unknown code returns 404.
- ☐ DynamoDB contains the mapping.
- ☐ CloudWatch logs show requests/errors without secrets.
- ☐ IAM denies unrelated DynamoDB actions.
- ☐ API throttling and request-size limits are considered.

7. Troubleshooting

Lambda 500: inspect CloudWatch logs; verify TABLE_NAME, Region, table name, and IAM policy.

AccessDeniedException: confirm the Lambda role permits required actions on the correct table ARN.

API Gateway 502: check Lambda integration, invoke permission, payload format, and response shape.

Incorrect short URL: verify BASE_URL includes the correct API stage and has no trailing slash.

8. Cost and availability notes

API requests, Lambda duration, DynamoDB reads/writes/storage, CloudWatch logs, and optional S3 usage may incur charges. Check current AWS pricing and create a budget alert.

A single-region serverless design can scale, but it does not by itself provide multi-region disaster recovery. Describe availability based on what you actually configure and test.

9. LinkedIn post template

❑ Built a Serverless URL Shortener on AWS!

I created a serverless URL-shortening API using Amazon API Gateway, AWS Lambda, and Amazon DynamoDB.

❑ Services used:

- API Gateway — exposes HTTP endpoints
- AWS Lambda — validates requests and handles URL logic
- DynamoDB — stores short-code mappings
- IAM — applies least-privilege permissions
- CloudWatch — logs, metrics, and troubleshooting
- Amazon S3 — optional storage for static assets or exports

❑ What I practiced:

- ✓ Building API-driven serverless workflows
- ✓ Persisting data with DynamoDB
- ✓ Securing access with IAM roles
- ✓ Monitoring and debugging with CloudWatch
- ✓ Testing API responses and redirects

This project strengthened my AWS serverless and cloud development skills.

#AWS #Serverless #AWSLambda #APIGateway #DynamoDB #CloudComputing #CloudProjects

10. Official AWS documentation

API Gateway: <https://docs.aws.amazon.com/apigateway/>

Lambda: <https://docs.aws.amazon.com/lambda/>

DynamoDB: <https://docs.aws.amazon.com/amazondynamodb/>

IAM: <https://docs.aws.amazon.com/IAM/latest/UserGuide/>

CloudWatch: <https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/>

S3: <https://docs.aws.amazon.com/AmazonS3/latest/userguide/>

AWS Pricing Calculator: <https://calculator.aws/>